MASTER 1 · UNITY / C#

Raycasting & Occlusion

Vision du garde · Ligne de vue · Session 04

Distance → FOV dot → Raycast → décision

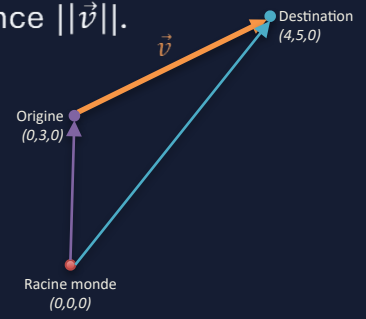
Le garde sait déjà regarder devant lui

Aujourd'hui, on retire l'illusion : un mur doit réellement interrompre la perception.

01 Direction

`toTarget = target - origin`

Un vecteur $\vec{v} = \hat{v}\ ||\vec{v}||$
- la direction unitaire $\hat{v}$
- la taille/distance $||\vec{v}||$.



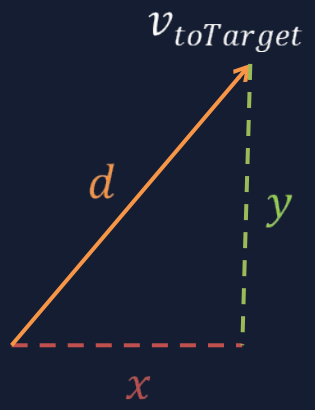
$$\overrightarrow{vecteur} = \overrightarrow{destination} - \overrightarrow{origine}$$
$$\vec{v} = (4,5,0) - (0,3,0)$$
$$\vec{v} = (4,2,0)$$

02 Limite simple en 1er :

SqrMagnitude < MaxDistance^2

= potentiellement visible ***(à 360° !)***

Pythagore :
$$d^2 = x^2 + y^2$$



Éviter les racines $\sqrt{\ }$:

$$||v_{toTarget}|| = d = \sqrt{x^2 + y^2}$$

$$d^2 = x^2 + y^2$$

03 Limites plus complexe en 2nd

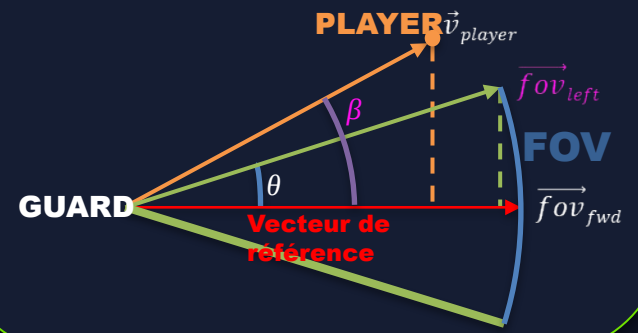
a. Produit scalaire $dot(\widehat{fov}_{fwd}, \hat{v})$

Compare l'alignement sans trigonométrie
et donc sans $\beta = \boldsymbol{Vector3.Angle}(\hat{v}, \widehat{fov}_{fwd})$

$$\text{proj} = \|\hat{v}\| \cos(\beta)$$
$$\text{proj} = \cos(\beta)$$

b. Comparaison avec limites FOV

$$\text{ref} = dot(\widehat{fov}_{fwd}, \widehat{fov}_{left})$$
$$\text{ref} = \|\widehat{fov}_{left}\| \cos(\theta) = \cos(\theta)$$



PROBLÈME

Une vision sans occlusion est un wallhack

Question de production : quel segment de monde devons-nous prouver libre ?



CONTRAT

Le joueur est détecté si :

proche ET dans le FOV **ET aucun obstacle**

ANATOMIE

Un Raycast est une requête bornée

Il ne “voit” pas. Il teste un rayon, une longueur, des catégories de collision.

SIGNATURE

```
Physics.Raycast(
     origin,
     direction,
     out hit,
     maxDistance,
     layerMask
)
```

SIGNIFICATION

Origin = œil
Direction = de l’œil vers la cible
out Hit = info sur le premier collider rencontré
Max distance = à quelle distance arrêter le rayon
Mask = ce qui peut bloquer = détectable par le rayon

type LayerMask

LAYERMASK

Le masque est une politique de visibilité

Choisir les bits, c’est définir ce qui a le droit de bloquer.

layers	0	1	2	3	4
mask	0	0	1	0	0

INCLUS

VisionOccluder

EXCLUS

Player

RÈGLE

Le masque décrit les blockers

PIPELINE

L’ordre des tests est le design

Les tests arithmétiques éliminent avant d’appeler la physique.



LIVE CODING

Construire la preuve, pas le booléen

Chaque branche doit être explicable et observable.

1 · CONFIGURER

target
viewDistance
fieldOfView
occluderMask
eyeHeight

2 · VALIDER

null → false

distance²

normalizations sûre
si $d^2 < \varepsilon$ → false/STOP
avec ε = 0,0001 par exemple

dot product

3 · PROUVER

Raycast borné

Hit = blocker

Debug.DrawLine

Gizmos

FLOOR

Mission : bloquer la vision à travers les murs

Une scène minimale, quatre tests, aucune magie.

SCÈNE

1 garde · 1 Player
2 murs + 1 gap
Colliders configurés
Layer `VisionOccluder`
Player hors du mask

ACCEPTANCE

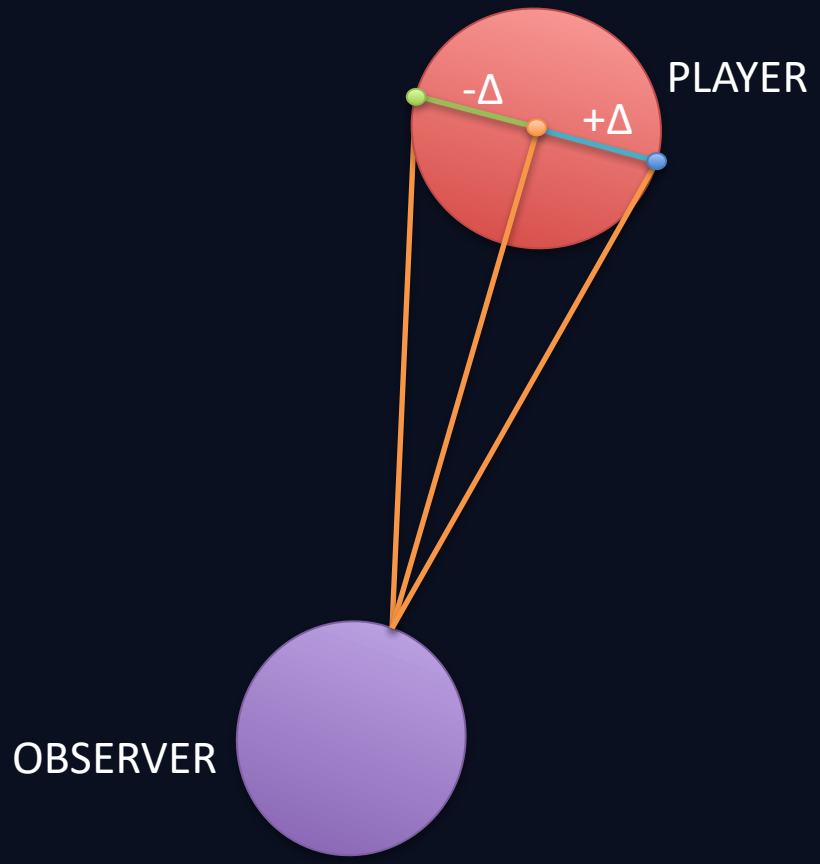
A dans champs de vision non obstrué
B derrière le garde
C hors portée
D dans champs de vision mais obstrué par mur

A vrai
B/C/D faux

POWERUP 1

Raycast sweep multi-angles

Le rayon unique devient un échantillonnage du cône.



SPEC

N ≥ 3 rayons
−FOV/2 → +FOV/2
nearest hit par sample

Visualiser (DrawRay) avant de mailler.

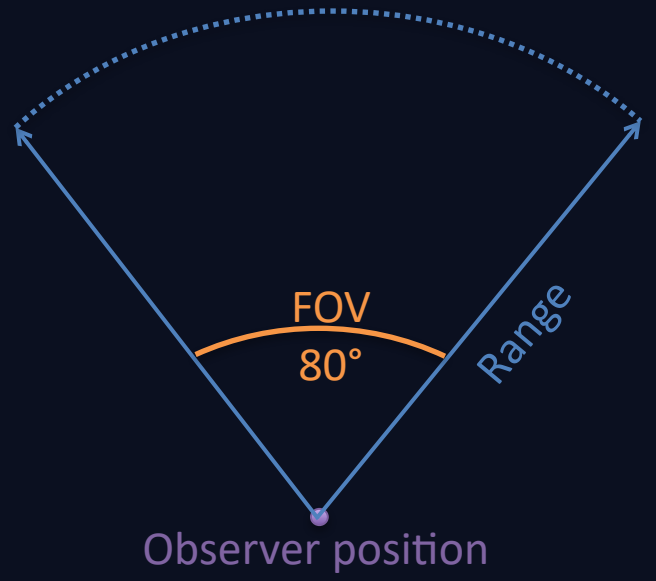
POWERUP 2

Procedural Vision Mesh

Le mesh visualise. Le pipeline Floor reste l’autorité gameplay.

INPUT

Observer position
FOV
Range
#Rays or #Triangles



POWERUP 2

Procedural Vision Mesh

Le mesh visualise. Le pipeline Floor reste l’autorité gameplay.

INPUT

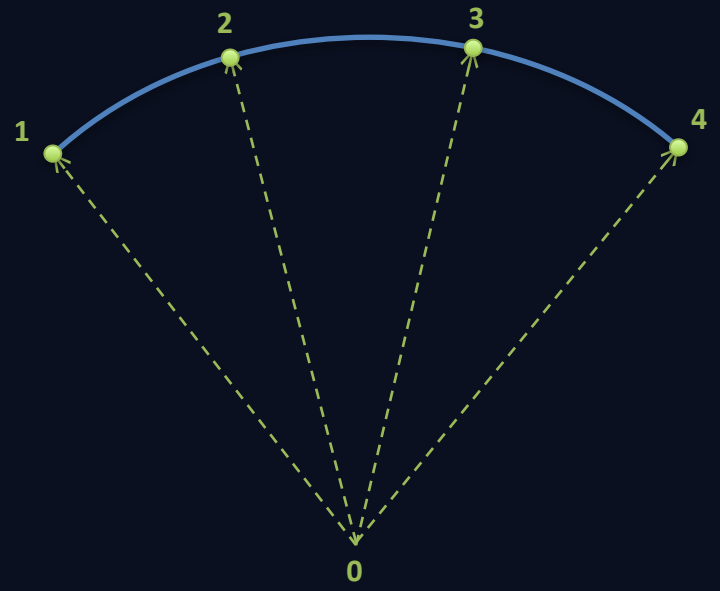
#Rays → #Triangles = #Rays -1

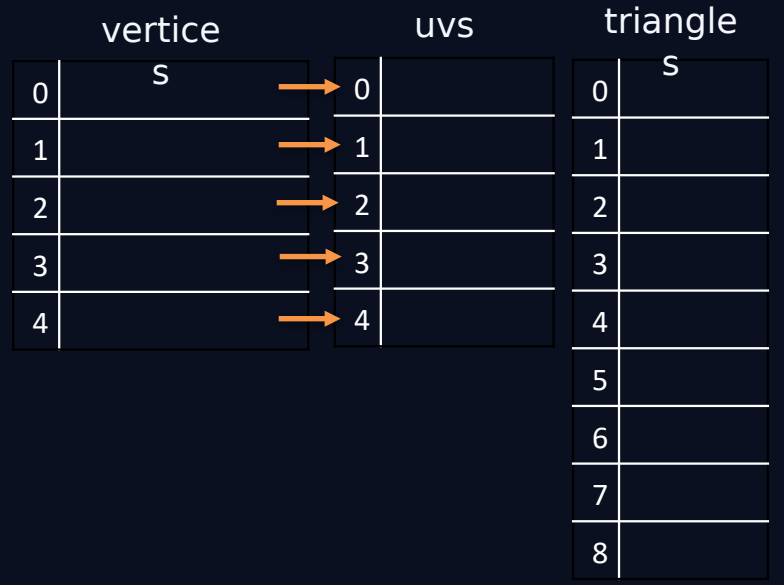
#Vertices = #Rays + origin

PREPARE when needed

vertices [# Δ + 2]
uvs [# Δ + 2]
triangles [# Δ * 3]

➔ 0 allocation en boucle update
Pour éviter le GC qui en suit





POWERUP 2

Procedural Vision Mesh

Le mesh visualise. Le pipeline Floor reste l’autorité gameplay.

INPUT

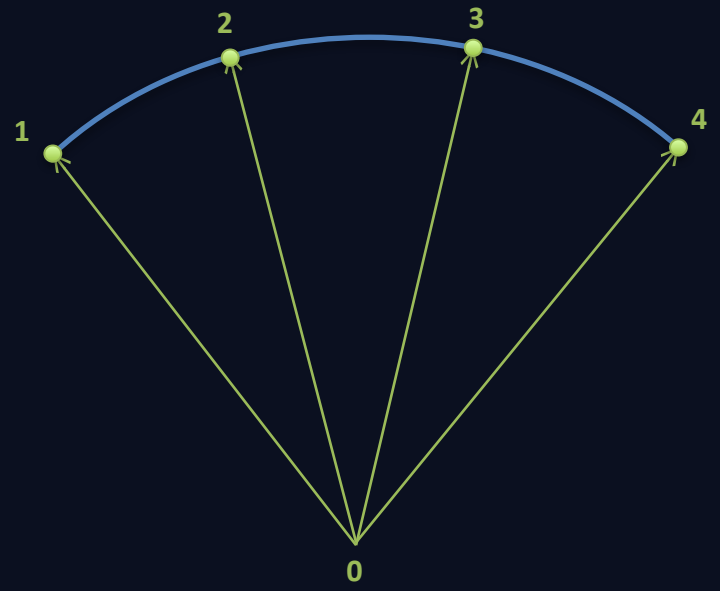
PREPARE when needed

SAMPLING

Lancer un rayon par vertex périphérique

Remplir **Vertices**[] avec :
si hit : hit.point
si rien : Range suivant dir rayon

/!\ Attention de bien convertir WS→OS



	vertices
0	(0 , 0 , 0)
1	(-10 , 0 , 15)
2	(-5 , 0 , 18)
3	(5 , 0 , 18)
4	(10 , 0 , 15)

	uvs
0	(0.5 , 0.5)
1	(0.2 , 0,7)
2	(0.4 , 0,8)
3	(0.6 , 0,8)
4	(0.7 , 0,7)

	triangles
0	
1	
2	
3	
4	
5	
6	
7	
8	

POWERUP 2

Procedural Vision Mesh

Le mesh visualise. Le pipeline Floor reste l’autorité gameplay.

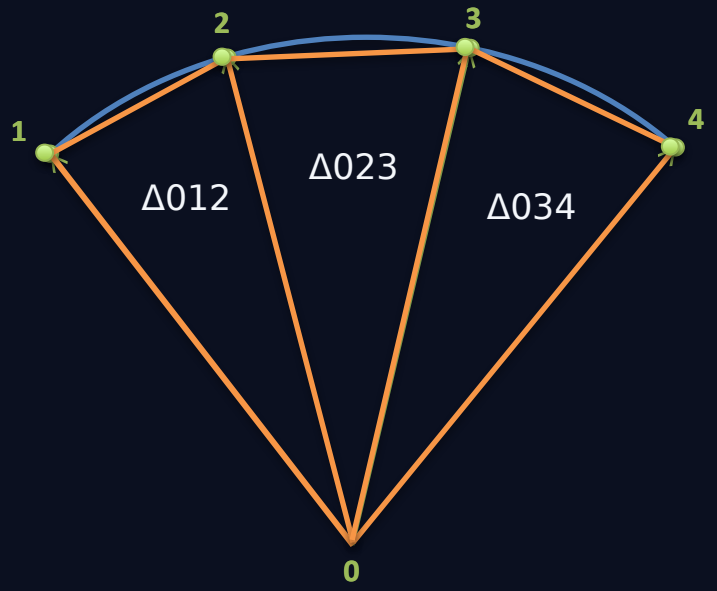
INPUT

PREPARE when needed

SAMPLING

GEOMETRY

Éventail de vertices
> **ordered** triangles
> utiliser méthodes **Set**
> Terminer par **RecalculateBounds**()



	vertices
0	(0 , 0 , 0)
1	(-10 , 0 , 15)
2	(-5 , 0 , 18)
3	(5 , 0 , 18)
4	(10 , 0 , 15)

	uvs
0	(0.5 , 0.5)
1	(0.2 , 0,7)
2	(0.4 , 0,8)
3	(0.6 , 0,8)
4	(0.7 , 0,7)

	triangles
0	0
1	1
2	2
3	0
4	2
5	3
6	0
7	3
8	4

Indices du tableau **vertices** (par groupe de 3)

POWERUP 2

Procedural Vision Mesh

Le mesh visualise. Le pipeline Floor reste l’autorité gameplay.

INPUT

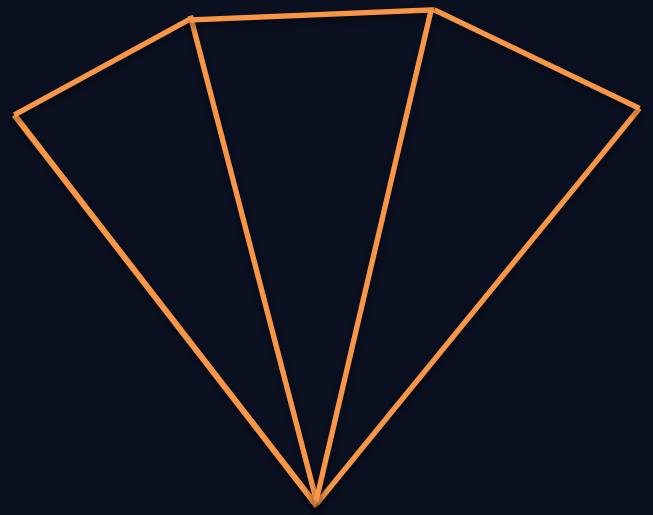
PREPARE when needed

SAMPLING

GEOMETRY

OUTPUT

MeshFilter
MeshRenderer
material calm/detected



DEBUG

Quand ça casse, suivre la chaîne

Ne pas “augmenter la portée”. Identifier la première étape fausse.

SYMPTÔME → QUESTION

Toujours bloqué → Player dans le mask ?

Jamais bloqué → mur/collider/layer ?

Derrière détecté → signe du vecteur ?

Flicker → seuil / précision ?

INSTRUMENTATION

LastDistance
LastAlignment
LastRayBlocked

Couleur :
vert = calme
rouge = détecté
jaune = bloqué

TAKEAWAY

Une détection est une conjonction

Distance dit “assez près”. Dot dit “assez devant”. Raycast dit “rien ne bloque”.

$$\texttt{near} \;\wedge\; \texttt{inFOV} \;\wedge\; \texttt{clearLineOfSight}$$

Construisez la preuve.
Puis rendez-là visible.

Floor d’abord · Powerups si logique de détection est validé